

CHAPTER 10

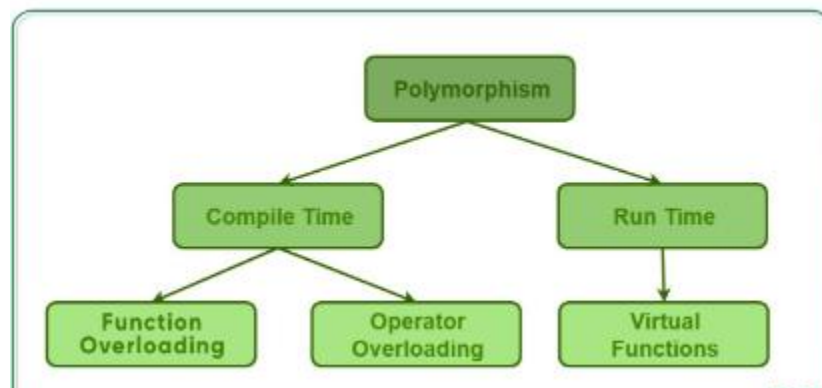
POLYMORPHISM

Polymorphism

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism, a person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism. Polymorphism is considered as one of the important features of Object Oriented Programming.

In C++ polymorphism is mainly divided into two types:

- Compile time Polymorphism
- Runtime Polymorphism



- **Compile time polymorphism:** The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding.

Function Overloading

When there are multiple functions with same name but different parameters then these functions are said to be **overloaded**. Functions can be overloaded by **change in number of arguments** or/and **change in type of arguments**. **You cannot overload function declarations that differ only by return type.**

Following is the example where same function **print()** is being used to print different data types

```

#include <iostream>
using namespace std;

class printData {
public:
    void print(int i) {
        cout << "Printing int: " << i << endl;
    }
    void print(double f) {
        cout << "Printing float: " << f << endl;
    }
    void print(char* c) {
        cout << "Printing character: " << c << endl;
    }
};

int main(void) {
    printData pd;

    // Call print to print integer
    pd.print(5);

    // Call print to print float
    pd.print(500.263);
    // Call print to print character
    pd.print("Hello C++");

    return 0;
}

```

When the above code is compiled and executed, it produces the following result –

```

Printing int: 5
Printing float: 500.263
Printing character: Hello C++

```

How Function Overloading Work?

- *Exact match:-* (Function name and Parameter)
- *If a not exact match is found:-*
 - >Char, Unsigned char, and short are promoted to an int.
 - >Float is promoted to double
- *If no match found:*
 - >C++ tries to find a match through the standard conversion.

Member function declarations with the same name and the name parameter-type-list cannot be overloaded if any of them is a static member function declaration. For example, following program fails in compilation.

```
#include<iostream>
class Test {
    static void fun(int i) {}
    void fun(int i) {}
};
```

```
int main()
{
    Test t;
    return 0;
}
```

```
// C++ program for function overloading
#include <iostream>
```

```
using namespace std;
class Sample
{
    public:
```

```
    // function with 1 int parameter
    void func(int x)
    {
        cout << "value of x is " << x << endl;
    }
```

```
    // function with same name but 1 double parameter
    void func(double x)
    {
        cout << "value of x is " << x << endl;
    }
```

```
    // function with same name and 2 int parameters
    void func(int x, int y)
    {
        cout << "value of x and y is " << x << ", " << y << endl;
    }
};
```

```
int main() {
```

```
    Sample obj1;
```

```
    // Which function is called will depend on the parameters passed
    // The first 'func' is called
```

```

obj1.func(7);

// The second 'func' is called
obj1.func(9.132);

// The third 'func' is called
obj1.func(85,64);
return 0;
}

```

Output:

```

value of x is 7
value of x is 9.132
value of x and y is 85, 64

```

In the above example, a single function named *func* acts differently in three different situations which is the property of polymorphism.

Operator Overloading

C++ also provide option to overload operators. For example, we can make the operator ('+') for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator '+' when placed between integer operands , adds them and when placed between string operands, concatenates them.

Example:

```

// CPP program to illustrate
// Operator Overloading
#include<iostream>
using namespace std;

class Complex {
private:
    int real, imag;
public:
    Complex(int r = 0, int i =0) {real = r;  imag = i;}

    // This is automatically called when '+' is used with
    // between two Complex objects
    Complex operator + (Complex const &obj) {
        Complex res;
        res.real = real + obj.real;
        res.imag = imag + obj.imag;
        return res;
    }
}

```

```

    void print() { cout << real << " + i" << imag << endl; }
};

int main()
{
    Complex c1(10, 5), c2(2, 4);
    Complex c3 = c1 + c2; // An example call to "operator+"
    c3.print();
}

```

Output:

12 + i9

In the above example the operator ‘+’ is overloaded. The operator ‘+’ is an addition operator and can add two numbers(integers or floating point) but here the operator is made to perform addition of two imaginary or complex numbers.

Runtime polymorphism

Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at	It is more flexible as all the things execute at

the compile time.

the run time.

Function Overriding

Now, let's consider the case where function name and prototype is same.

```
class A                // base class declaration.
{
    int a;
    public:
    void display()
    {
        cout<< "Class A ";
    }
};

class B : public A      // derived class declaration.
{
    int b;
    public:
    void display()
    {
        cout<<"Class B";
    }
};
```

In the above case, the prototype of *display()* function is the same in both the **base and derived class**. Therefore, the static binding cannot be applied. It would be great if the appropriate function is selected at the run time. This is known as **run time polymorphism**.

Function overriding occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be **overridden**.

```
#include <iostream>
using namespace std;
class base
{
    public:
    virtual void print ()
    { cout<< "print base class" <<endl; }
    void show ()
    { cout<< "show base class" <<endl; }
};

class derived:public base
{
```

```

public:
    void print () //print () is already virtual function in derived class, we could also declared as
virtual void print () explicitly
    { cout<< "print derived class" <<endl; }

    void show ()
    { cout<< "show derived class" <<endl; }
};

//main function
int main()
{
    base *bptr;
    derived d;
    bptr = &d;

    //virtual function, binded at runtime (Runtime polymorphism)
    bptr->print();

    // Non-virtual function, binded at compile time
    bptr->show();

    return 0;
}

```

Output:

```

print derived class
show base class

```

Late binding or Dynamic linkage

In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

C++ virtual function

Virtual function plays an important role in object oriented programming. Ordinary function uses the concept of early binding (compile time). However, a virtual function uses the concept of late binding. Mainly pointer plays a vital role in virtual function. A virtual function is one that does not really exist but it appears real in some parts of a program. When a virtual function is used in

a C++ program, then the program which calls a function of one class, actually in real matters can call a function of different class. A virtual function is a function that is declared as virtual in base class. Here we use the pointer to base class to refer to all the derived class objects.

When we use the same function name in both the base and derived classes, the function in base class is declared as virtual using the keyword “virtual” preceding its normal declaration. In other words, the use of same function in both the base and derived classes can be done in C++ programming, but for this purpose a function in the base class must be declared as virtual. Such type of function is called **virtual function**.

So after the declaration of virtual function it is the C++’s job to find which function to use at run time based on the type of object pointed to by the base class pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects, we can execute different versions of the virtual function.

Definition: Virtual functions are public member functions, used for special purpose. These are defined in base class and again are redefined in the derived class i.e. virtual functions are defined both in the base and derived class with the same name.

Virtual Function employs the concept of late binding by allocating memory space during execution time not during compilation time.

Rules of Virtual Function

1. Since we already specify the member function to be a virtual function in the base class with the help of the “virtual” keyword. Now we need not again specify it while defining it in the derived class.
2. A member function of the base class cannot be a virtual function and a friend function simultaneously.
3. A member function of the base class cannot be a virtual function with the keyword static.
4. We can access virtual functions only with the help of a pointer or a reference.
5. The basic model of the virtual function in the base class should be similar while redefining it in the derived class.
6. It is not necessary to redefine the virtual function in the derived class. If the virtual function is not overridden, then we can use the original function declared or defined in the base class.
7. It is not possible to introduce a virtual constructor inside the class, although it is possible to introduce a virtual destructor.

Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
using namespace std;
```

```

class A
{
    int x=5;
    public:
    void display()
    {
        std::cout << "Value of x is : " << x<<std::endl;
    }
};
class B: public A
{
    int y = 10;
    public:
    void display()
    {
        std::cout << "Value of y is : " <<y<< std::endl;
    }
};
int main()
{
    A *a;
    B b;
    a = &b;
    a->display();
    return 0;
}

```

Output:

undefined reference to `WinMain

Above code shows error because display() function must be virtual in this case. We can invoke derived class function with base class pointer only in case of virtual function. Hence display() function in base class must be declared virtual.

```

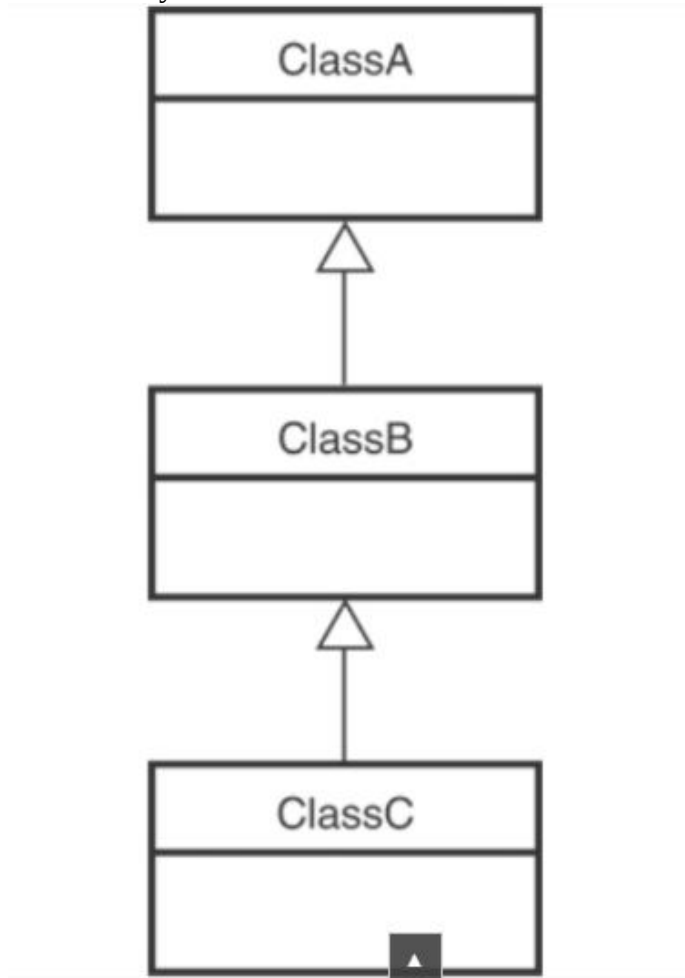
#include <iostream>
using namespace std;
class A
{
    int x=5;
    public:
    Virtual void display()
    {
        std::cout << "Value of x is : " << x<<std::endl;
    }
};
class B: public A
{
    int y = 10;
    public:
    void display()
    {

```

```
        std::cout << "Value of y is : " <<y<< std::endl;
    }
};
int main()
{
    A *a;
    B b;
    a = &b;
    a->display();
    return 0;
}
```

Definitions:

1. A **virtual** function is a *member* function which is declared in the *base* class using the keyword `virtual` and is re-defined (Overriden) by the *derived* class.
2. The term **Polymorphism** means the ability to take many forms. It occurs if there is a hierarchy of classes which are all related to each other by *inheritance*.



Consider the following simple program as an example of runtime polymorphism. **The main thing to note about the program is that the derived class's function is called using a base class pointer.**

The idea is that virtual functions are called according to the type of the object instance pointed to or referenced, not according to the type of the pointer or reference.

In other words, virtual functions are resolved late, at runtime.

Now, we'll look at an example using both these concepts to clarify your understanding.

```
#include <iostream>
using namespace std;

// Base class
class Shape
{
public:
    Shape(int l, int w)
    {
        length = l;
        width = w;
    } // default constructor
    int get_Area()
    {
        cout << "This is call to parent class area" << endl;
        return 1;
    }

protected:
    int length, width;
};

// Derived class
class Square : public Shape
{
public:
    Square(int l = 0, int w = 0)
        : Shape(l, w)
    {
    } // declaring and initializing derived class
    // constructor
    int get_Area()
    {
        cout << "Square area: " << length * width << endl;
    }
};
```

```

        return (length * width);
    }
};
// Derived class
class Rectangle : public Shape
{
public:
    Rectangle(int l = 0, int w = 0)
        : Shape(l, w)
    {
    } // declaring and initializing derived class
    // constructor
    int get_Area()
    {
        cout << "Rectangle area: " << length * width
              << endl;
        return (length * width);
    }
};

int main(void)
{
    Shape* s;
    Square sq(5, 5); // making object of child class Square
    Rectangle rec(4, 5); // making object of child class Rectangle

    s = &sq;
    s->get_Area();
    s = &rec;
    s->get_Area();

    return 0;
}

```

Output

This is call to parent class area

This is call to parent class area

In the above function:

- we store the address of each child class **Rectangle** and **Square** object in **s** and
- then we call the **get_Area()** function on it,
- ideally, it should have called the respective **get_Area()** functions of the child classes but
- instead it calls the **get_Area()** defined in the base class.

- This happens due static linkage which means the call to **get_Area()** is getting set only once by the compiler which is in the base class.

Let's see a simple example of run time polymorphism using virtual function.

```
#include <iostream>
using namespace std;
class Shape
{
    // base class
public:
    virtual void draw()
    {
        // virtual function
        cout<<"drawing..."<<endl;
    }
};
class Rectangle: public Shape    // inheriting Shape class.
{
public:
    void draw()
    {
        cout<<"drawing rectangle..."<<endl;
    }
};
class Circle: public Shape      // inheriting Shape class.
{
public:
    void draw()
    {
        cout<<"drawing circle..."<<endl;
    }
};
int main(void)
{
    Shape *s;           // base class pointer.
    Shape sh;           // base class object.
    Rectangle rec;
    Circle cir;
    s=&sh;
    s->draw();
    s=&rec;
    s->draw();
    s=&cir;
}
```

```
s->draw();  
return(0);  
}
```

Output

drawing...

drawing rectangle...

drawing circle...

What is the use?

Virtual functions allow us to create a list of base class pointers and call methods of any of the derived classes without even knowing kind of derived class object.

Pure Virtual Function

A pure virtual function is a virtual function with no body[{ }]. So the body of the virtual function in the base can be removed. A pure virtual function is a type of function, which has only a function declaration. Here base class becomes an abstract class. These functions are do-nothing virtual functions. Another name of pure virtual function is empty virtual function. A pure virtual function can be declared by equating it to zero.

A general syntax for the declaration of pure virtual function is:

virtual return-type function name() = 0;

For example, the declaration of a pure virtual function is as :

virtual display() = 0;

It is a pure virtual function declaration. A pure virtual function is a function declared in a base class that has no definition relative to the base class. In such cases, the compiler requires each derived class to either define the function or re-declared it as a pure virtual function. So a class containing pure virtual functions cannot be used to declare any objects of its own. In the base class, a pure virtual function serves no purpose except to act as polymeric interface for the derived class. After declaring the pure virtual function, there is no need to create an object of base class because it will be meaningless to call a function with no body.

Note that if you define a pure virtual function in the base class, the either declare it in the derived class or redefine that function as a virtual function.

Abstract Classes

An abstract class is one that is not used to create object. A class that declared a pure virtual function is called an **abstract class** and a class that does not define a pure virtual function is called **concrete class**. Abstract class can declare pointers for manipulating the objects of the derived classes. An abstract class is used as a base class for deriving. Specific classes of the same kind.

It defines properties common to other classes derived from it. Since an Abstract class contains one or more functions for which there is no definition, no objects can be created using an abstract class, however, only pointers to an abstract class can be created. An Abstract class not create any instances.

Always Remember:

1. Creating an object of an abstract class create error.
2. An abstract class contains one or more pure virtual functions that derived classes should override.

Sample of Abstract class having pure virtual function

```
// An abstract class
class Test
{
    // Data members of class
public:
    // Pure Virtual Function
    virtual void show() = 0;

    /* Other members */
};
```

A pure virtual function is implemented by classes which are derived from a Abstract class.

Following is a simple example to demonstrate the same.

```
#include<iostream>
using namespace std;

class Base
{
```

```

    int x;
public:
    virtual void fun() = 0;
    int getX() { return x; }
};

// This class inherits from Base and implements fun()
class Derived: public Base
{
    int y;
public:
    void fun() { cout << "fun() called"; }
};

int main(void)
{
    Derived d;
    d.fun();
    return 0;
}

```

Points to be Remember

1. A class is abstract if it has at least one pure virtual function.

```

#include<iostream>
using namespace std;
class Test
{
    int x;
public:
    virtual void show() = 0;
    int getX() { return x; }
};

int main(void)
{
    Test t;
    return 0;
}

```

:

Output:

Compiler Error: cannot declare variable 't' to be of abstract type 'Test' because the following virtual functions are pure

within 'Test': note: virtual void Test::show()

2. We can have pointers and references of abstract class type.

For example the following program works fine.

```
#include<iostream>
using namespace std;

class Base
{
public:
    virtual void show() = 0;
};

class Derived: public Base
{
public:
    void show() { cout << "In Derived \n"; }
};

int main(void)
{
    Base *bp = new Derived;
    bp->show();
    return 0;
}
```

Output:

In Derived

3. If we do not override the pure virtual function in derived class, then derived class also becomes abstract class.

The following example demonstrates the same.

```
#include<iostream>
using namespace std;
class Base
{
public:
    virtual void show() = 0;
```

```
};

class Derived : public Base { };
```

```
int main(void)
{
    Derived d;
    return 0;
}
```

Compiler Error: cannot declare variable 'd' to be of abstract type 'Derived' because the following virtual functions are pure within 'Derived': virtual void Base::show()

4) ***An abstract class can have constructors.***

For example, the following program compiles and runs fine.

```
#include<iostream>
using namespace std;

// An abstract class with constructor
class Base
{
protected:
int x;
public:
virtual void fun() = 0;
Base(int i) {
    x = i;
    cout<<"Constructor of base called\n";
}
};

class Derived: public Base
{
    int y;
public:
    Derived(int i, int j):Base(i) { y = j; }
    void fun() { cout << "x = " << x << ", y = " << y<<'\n'; }
};

int main(void)
{
    Derived d(4, 5);
```

```

    d.fun();

//object creation using pointer of base class
Base *ptr=new Derived(6,7);
    ptr->fun();
    return 0;
}

```

Output:

Constructor of base called

x = 4, y = 5

Constructor of base called

x = 6, y = 7

Virtual Destructors

Virtual Destructor in C++ is a member function that **frees up the memory allocated by the object of a child class or derived class when it is removed from the memory using the parent class pointer object**. Virtual destructor in C++ is mainly responsible for resolving the memory leak problem.

Deleting a derived class object using a pointer of base class type that has a non-virtual destructor results in undefined behavior. To correct this situation, the base class should be defined with a virtual destructor.

For example, the following program results in undefined behavior.

```

// CPP program without virtual destructor
// causing undefined behavior
#include <iostream>

using namespace std;

class base {
public:
    base()
    { cout << "Constructing base\n"; }
    ~base()
    { cout<< "Destructing base\n"; }
};

class derived: public base {
public:
    derived()

```

```

        { cout << "Constructing derived\n"; }
    ~derived()
        { cout << "Destructing derived\n"; }
};

```

```

int main()
{
    derived *d = new derived();
    base *b = d;
    delete b;
    getchar();
    return 0;
}

```

```

// CPP program without virtual destructor
// causing undefined behavior
#include <iostream>

```

```

using namespace std;

```

```

class base {
public:
    base()
        { cout << "Constructing base\n"; }
    ~base()
        { cout<< "Destructing base\n"; }
};

```

```

class derived: public base {
public:
    derived()
        { cout << "Constructing derived\n"; }
    ~derived()
        { cout << "Destructing derived\n"; }
};

```

```

int main()
{
    derived *d = new derived();
    base *b = d;
    delete b;
    getchar();
    return 0;
}

```

Output

```
Constructing base  
Constructing derived  
Destructing base
```

As we can see in the above output, when the compiler compiles the code, it calls a pointer object in the main function that refers to the base class. So, it executes the base class' constructor() function and then moves to the derived class' constructor() function. After that, it deletes the pointer object occupied by the base class' destructor and the derived class' destructor. The base class pointer only removes the base class's destructor without calling the derived class' destructor in the program. Hence, it **leaks the memory** in the program.

What are the reasons for memory leaks?

The major reasons for memory leaks are as follows:

- When the user forgets to delete data that has been dynamically allocated.
- When the user fails to notice that the code may bypass a delete statement in unforeseen situations.
- And, when the user assigns the result of a new statement to a pointer that was already pointing to an allocated object.

Note: If the base class destructor does not use a virtual keyword, only the base class destructor will be called or deleted its occupied space because the pointer object is pointing to the base class. So it does not call the derived class destructor to free the memory used by the derived class, which leads to memory leak for the derived class.

Making base class destructor virtual guarantees that the object of derived class is destructed properly, i.e., both base class and derived class destructors are called. For example,

```
// A program with virtual destructor  
#include <iostream>  
  
using namespace std;  
  
class base {  
public:  
    base()  
    { cout << "Constructing base\n"; }  
    virtual ~base()  
    { cout << "Destructing base\n"; }  
};
```

```

class derived : public base {
public:
    derived()
    { cout << "Constructing derived\n"; }
    ~derived()
    { cout << "Destructing derived\n"; }
};

int main()
{
    derived *d = new derived();
    base *b = d;
    delete b;
    getchar();
    return 0;
}

```

Output:

```

Constructing base
Constructing derived
Destructing derived
Destructing base

```

Note:

1. It is a syntax error to declare a constructor as virtual.
2. One must use a virtual destructor in class, having virtual functions, even if these are not required, because classes derived from this class may have destructors.

Binding

Binding concept is linked with polymorphism and helps in creating virtual function. There are two types of Binding used in object-oriented programming. These are:

1. Late Binding or Dynamic Binding
 2. Early Binding or Static Binding
1. Late Binding or Dynamic Binding is implemented in C++ through the use of virtual array having pointers or virtual function. It is compiled by compilers for each class. Dynamic Binding helps to modify and extend the function of your program. Dynamic Binding can provide a library of classes, which can be used by other programmer without source code.

When you select a function during execution time, then such type of linkage is called Dynamic linkage. It is also called Dynamic Binding or late Binding. In Dynamic Binding, an object of a class must be declared either as a pointer to a class or a reference to a class.

2. When a function is selected during compile time in the normal processing, then such type of linkage is called static linkage. It is also called Static Binding or Early Binding. By default, C++ always use the Early Binding. By using the early binding one can get the efficient results. Also function call processing becomes faster in early binding.

Note that during compilation time, the C++ compiler determines the function call on the basis of parameters passed to the function or the function return type. Compiler substitutes the correct function for each call.